



KRESZ alapú tanulás segítő játék fejlesztése Unity keretrendszerben

Készítette

Fodor Győző Benedek

Programtervező Informatikus Bsc.

Témavezető

Troll Ede Mátyás

Tanársegéd

EGER, 2026

Tartalomjegyzék

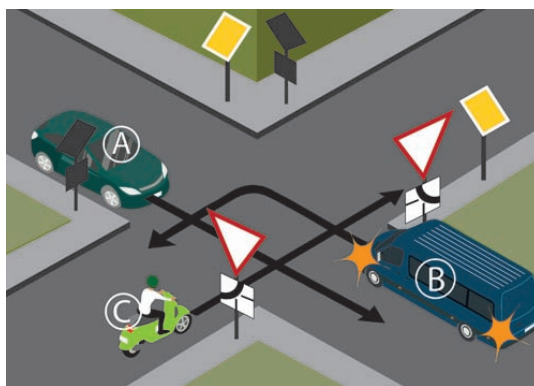
Bevezetés	4
1. Technológiai áttekintés	5
1.1. Unreal Engine	6
1.2. Godot	6
1.3. Unity	7
1.3.1. Komponensek	8
1.4. Közlekedési szabályok	9
2. Rendszerterv	10
2.1. A rendszer célja	10
2.2. Telepítés	10
2.3. Fejlesztéshez használt eszközök	10
2.4. Funkcionális elvárások	10
3. Játékmenet	12
3.1. Szabad játék	12
3.2. Teszt játék	13
4. Saját szoftver	15
4.1. Fejlesztési idővonal	15
4.1.1. Tanulási időszak	15
4.1.2. Szoftver alapok	15
4.1.3. Unity alapok	15
4.1.4. Legfrissebb verzió	16
4.2. Menü	16
4.3. Játékprogram	16
4.3.1. Utak	17
4.3.2. Járművek	18
4.3.3. Kereszteződések	19

5. Fejlesztési lehetőségek	20
5.1. Textúrák és modellek	20
5.2. Új pályák és szituációk	21
5.3. Új szabályrendszerek	21
5.4. Szabálylista	23
5.5. Testreszabható feladatok	23
6. Tesztelés	24
6.1. Manuális	24
6.2. Egységteszt	25
Összegzés	26
Irodalomjegyzék	27
Ábrajegyzék	28

Bevezetés

A szakdolgozatom témáját akkor kellett választanom, amikor a szabadidőm nagy részét a jogosítványom megszerzésére fordítottam. A szabályok megértése és megtanulása egyszerű volt, de az úton, a közlekedési helyzetekben való alkalmazásuk már sokkal bonyolultabb. Az úgynevezett „3 pontos” feladatok, amik egy KRESZ vizsga jelentős részét alkotják, erre alapulnak. Az ilyen feladványok játékos gyakorlására akartam megoldást találni.

Egy olyan felületet szerettem volna létrehozni ami magába tudja foglalni egy közlekedési szituáció minden elemét. Fontos hogy a részletek könnyen felismerhetőek legyenek, és az egész jelenet interaktív legyen, segítve az adott helyzetben érvényesülő szabályok megértését. A tervem megvalósítására a Unity keretrendszert találtam a legalkalmasabbnak.



1. ábra. Három pontos feladat
(szuperjogsi.hu) [9]

A szakdolgozat kódja, a hozzá tartozó build, és a játék működését bemutató videó a itt található:

Forráskód:

Build:

Videó:

1. fejezet

Technológiai áttekintés

A 20. század alkonyán a játékfejlesztők még sokkal kevesebben és kisebb erőforrásból dolgoztak, így sokkal gyakoribb volt, hogy a játékaikhoz saját motort fejlesztettek, mivel akkoriban nem sok kész, piaci játékmotor volt elérhető. Nagy segítség lehetett viszont, ha valaki meg tudott vásárolni egy kész kódbázist.

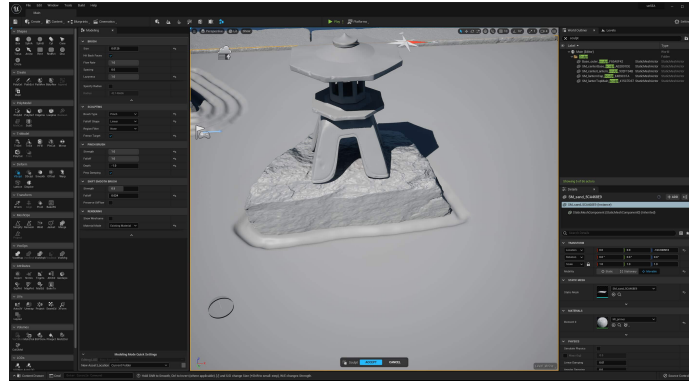
Ilyen volt például az *id Software* cég fejlesztette „id Tech” motor, melynek később több verziója is készült. Ezeket a motorokat használták az első meghatározó FPS (first person shooter) játékok mint például *DOOM*, *Quake*, *Wolfenstein* sorozatok alapjául.[1]



1.1. ábra. Korai képernyőkép a DOOM játékból (DOOM 1993) [15]

1.1. Unreal Engine

Az *Unreal Engine* az mai napig egyik legmeghatározóbb játékmotor. Az Epic Games játékfejlesztő cég 1998-ban készítette az *Unreal* nevű játékukhoz. Azóta 4-5 évente jelent meg új verzió, ami mindig jelentős változásokat hozott a aktuális játékiparba.



1.2. ábra. Unreal Engine kezelőfelület (unrealengine.com) [16]

Eredetileg az *Unreal Engine* is az FPS genre kiszolgálására készült. Később viszont a TPP (third person perspective) féle kalandjátékok és rengeteg más 3D-s játék alapjául is szolgált, legyen az lövöldözős, kaland, nyílt világ, vagy történet műfajú.

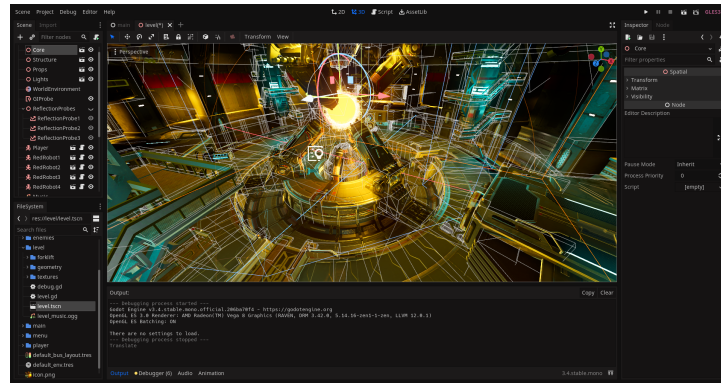
A motor a mai napig C++ nyelven működik, 2012-ig Java-val társítva az *UnrealScript*-et használta, azonban ezt lecserélték és azóta a *Blueprint* vizuális programozási felület a fejlesztők barkácsolóasztala.

Az alapvetően ingyenes szoftver legújabb, 2022-es verziójának forráskódja a GitHub-on is elérhető, viszont a kereskedelmi felhasználás jogdíj köteles, amit az Epic Games a bevétel 5%-aként határozott meg. [4]

1.2. Godot

Egy másik szintén ingyenesen hozzáférhető és népszerű játékmotor a *Godot*. *Juan Linietsky* fejlesztette és adta ki először 2001-ben. A motor több néven is ismerté vált, majd 2014-ben felkerült GitHubra. Napjainkban a fejlesztés a 4.0-ás verzióánál tart, ahol a 3D-s felületet igyekeznek felzárkóztatni a versenytársakhoz.

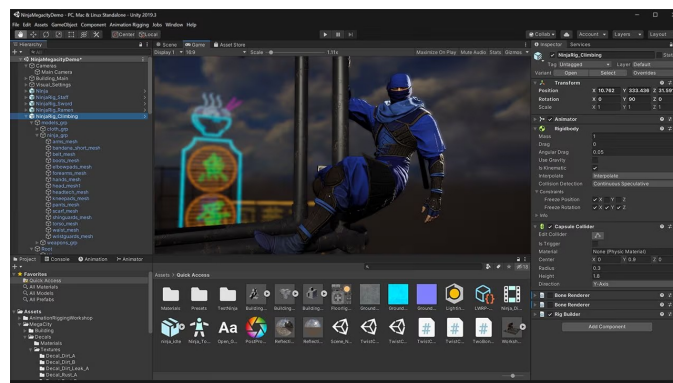
A motor nem csak egy programozási nyelvhez kötött, de a három leggyakoribb a C++ és C#, valamint a GDScript. A GDScript a Godot saját rendszeréhez optimalizált objektum orientált programozási nyelv, a Pythonhoz hasonló struktúrával. A vizuális programozás opciója itt is megjelenik, régebben a *VisualScript*-ként, ez viszont a 4.0-ás verzióban már nem elérhető. [3]



1.3. ábra. Godot kezelőfelület (Godot 4.6 2026) [17]

1.3. Unity

A Unity a *Unity Technologies* fejlesztői által készített ingyenesen használható játékmotor. A motor támogatása kiterjed a számítógépeken kívül a konzolokra és okos telefonokra is. A 2D-s és 3D-s alkalmazások készítésére is alkalmas fejlesztői környezet hamar elterjedt a mobilfejlesztők és a feltörekvő, egy vagy több személyes kis fejlesztőcsapatok körében. A Unity felépítése könnyen megérthető és a C# programozási nyelv is gyorsan tanulható, emellett rengeteg oktatóvideó és kurzus is segíti a kezdőket az alapok elsajátításában.[4]

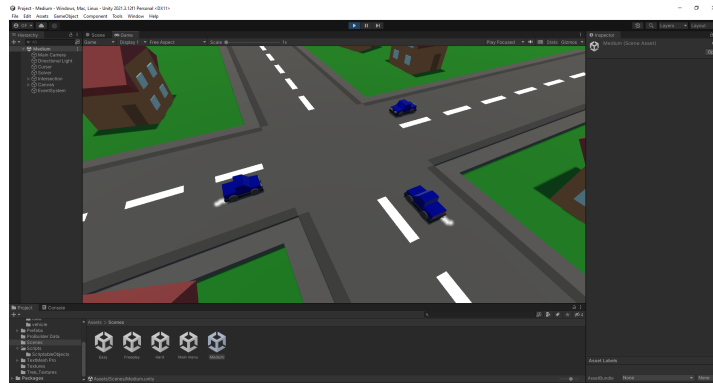


1.4. ábra. Unity kezelőfelület (2026) [18]

A fejlesztőcég először *Over The Edge Entertainment* jelent meg, csak 2007-ben nevezték át *Unity Technologies*-ra. [5]

A legjelentősebb fejlesztései:

- 2010: *Unity Asset Store*. Egy „Piac”, ahol a Unity-ben való fejlesztéshez használható fájlokat találhatsz ingyenesen és díjazva is.
- 2018: *Unity Hub*. Egy külön felület, ahol a Unity projekteket kezelheted és a motor különböző verziói közötti navigálást is megkönnyíti.



1.5. ábra. Unity kezelőfelület (Saját projekt)

1.3.1. Komponensek

A Komponensek a Unity egyik fő résztvevői. A komponens tulajdonságai teszik lehetővé, hogy egy *GameObject* személyre szabható legyen. A *GameObject*-ek építik fel a játékokat, ezek a fő elemei a kész programnak. Sok komponens mint például a *Transform* vagy a *Collider*-ek, kevesebb funkciót ágyaznak magukba vagy rengetegszer felbukkanhatnak a program során. [6]

A Unity a saját komponensek létrehozását is lehetővé teszi, ezáltal kivételes szabadságot adva a fejlesztőnek. Amennyiben ezeknek a komponenseknek őse a *MonoBehaviour* osztály, az összes többi *GameObject*-re rácsatolhatóvá válnak és így új funkciókat adhatunk az adott objektumnak. [7]

1.4. Közlekedési szabályok

A Közúti Rendelkezők Egységes Szabályozása (rövidítve: KRESZ) az a közlekedési szabálygyűjtemény amely a Magyarország területén levő közutakon és közforgalom elől el nem zárt magánutakon folyó közlekedést szabályozza.

A közúti forgalom rendjét már a 19. században elkezdték meghatározni. Az első magyar KRESZ egy 1910-ben született belügyminiszteri rendelet ami az azt egy évvel megelőző *Párizsi egyezmény* alapjaira készült. Ekkor még csak töredéke volt a mai szabályrendszernek. Ami viszont már ebben a korban és megjelent a KRESZ-ben az az autók műszaki vizsgálata és rendszámozása, a gépjárművezetők képzése, valamint az első négy közúti jelzőtábla:

- útkereszteződés
- kanyar
- vasúti átjáró
- bukkanó

1941-ben vezették be a jobboldali közúti közlekedést és ezzel együtt az úgynevezett „Jobbkéz szabályt”.



1.6. ábra. KRESZ táblák [10]

2. fejezet

Rendszerterv

2.1. A rendszer célja

A szakdolgozat célja egy olyan játék létrehozása, amiben a KRESZ vizsgák és tananyagokban szereplő „3-pontos” feladatokat játékos módon érthetőbbé, átláthatóbbá és tanulhatóbbá teszi. Fontos, hogy a játékos tudja tesztelni a tudását, a játék pedig interaktív és reszponzív legyen.

2.2. Telepítés

A nem szükséges telepíteni a játékszoftvert, egyből a kicsomagolás után indítható a mappában található alkalmazással.

2.3. Fejlesztéshez használt eszközök

- Unity: A játék készítéséhez használt fejlesztői környezet. Verzió: 2021.3.12f1
- Visual Studio: A programozói környezet, amely Unity támogatással rendelkezik. Verzió: Community 2022
- Visual Studio, WinForms: Vizuális fejlesztőkörnyezet a Visual Studio-n belül. A program kezdeti algoritmusainak a C# nyelvben való megvalósításra szolgált.

2.4. Funkcionális elvárások

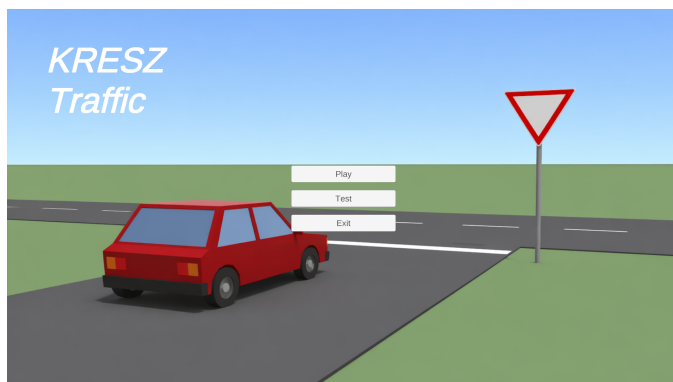
- KRESZ alapú szabályrendszer: Egy olyan háttér algoritmus ami a szituációban szereplő objektumokat kezelni. Az algoritmus áthaladási sorrendet állít fel ami egyben a feladvány helyes megoldását is meghatározza.

- Interaktív játékos: A feladatot alkotó szereplők kattinthatóak legyenek és rendelkezzenek animált mozgással. A játék értesítse a felhasználót a hibás lépésről, vagy helyes megoldásról.
- Szabad játék: Végtelen ismétlődő randomizált feladványok ahol a játékos gyakorolhatja a közlekedési szituációkat.
- Nehézségi szintek: Különböző nehézségű feladványok a könnyebben követhető tanulási és gyakorlási görbéhez valamint a monotonitás elkerüléséhez.
- Teszt felület: Egy véges „Teszt” mód ahol a gyakorolt feladatokat adott megadott összesített sikerességi rátával kell megoldani a teszt teljesítéséhez. Támpont a tudás és gyakorlat fejlődésében.
- Pontrendszer: Mind a szabad játéknak, mind a teszt felületnek megfelelő pontrendszer ami a játékos eredményét rögzíti folyamatosan.

3. fejezet

Játékmenet

Az indítás után az első megálló a főmenü. Itt választhatjuk ki, hogy a „Play” gombra nyomva egy szabad játékba kezdünk, vagy a „Teszt”-el egy tesztet indítunk, ahol megmérgethetjük saját tudásunkat a gyakoroltak alapján.



3.1. ábra. A játék Főmenüje

3.1. Szabad játék

A szabad játék egy végtelen játékmód. Itt az összes elérhető variációból véletlenszerűen kiválasztott feladatokat kapunk amiket a közlekedési szituációban résztvevő autókra kattintva kell megoldanunk. Ha helyes sorrendben választjuk ki az autókat, a feladatot teljesítettük, és léphetünk tovább a következőre.

Minden sikeresen megoldott feladat után egy pont jár, az ide tartozó pontrendszer a játékos aktuális játéka alatt helyesen megoldott feladványait tartja számon egész addig még a játékot be nem fejezik.



3.2. ábra. Hibásan megoldott feladat a szabad játékban

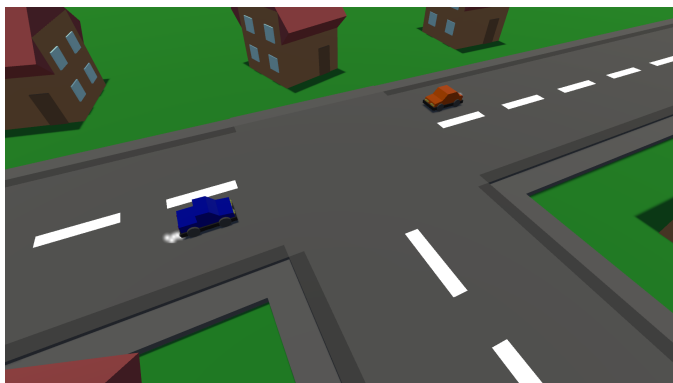


3.3. ábra. Sikeresen megoldott feladat a szabad játékban

3.2. Teszt játék

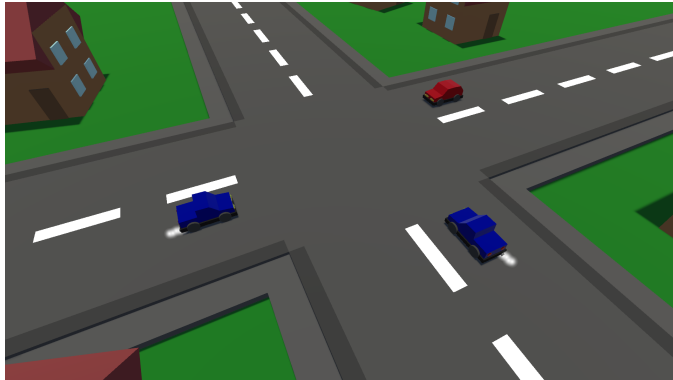
A „Teszt” gombra kattintva egy teszt játék veszi kezdetét. A *KRESZ vizsga* mintájára, adott darab feladványt kell megoldania a játékosnak. Egy teszt feladatai már meghatározottabbak mint a szabad játék situációi. Van több nehézségi szint. A különböző nehézségű feladatokat arányosan osztja el a rendszer, így a könnyebb és nehezebb situációkkal egyaránt meg kell tudni birkózni a teszt teljesítéséhez.

– Könnyű:



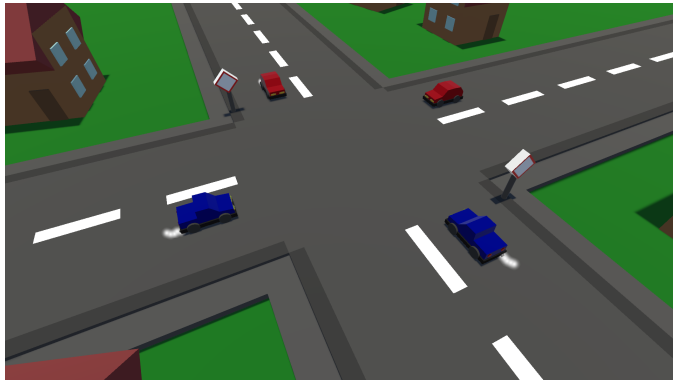
3.4. ábra. Könnyű szintű feladat

– Közepes:



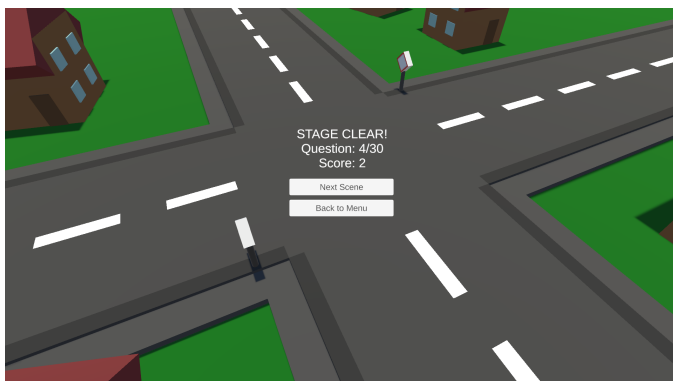
3.5. ábra. Közepes szintű feladat

– Nehéz:



3.6. ábra. Nehéz szintű feladat

Itt is van pontrendszer, de itt már nem csak a rekordok fenntartása végett. A teszt sikeres teljesítéséhez, az összesen megszerezhető maximális pontszám legalább 90%-át kell elérnie a játékosnak, de persze mindig lehet törekedni a tökéletes eredményre.



3.7. ábra. A Teszt játék köztes menüje egy sikeres feladatmegoldás után

4. fejezet

Saját szoftver

4.1. Fejlesztési idővonal

A programom teljes kifejlesztése összesen 2 évet vett igénybe. Ez idő alatt voltak aktívabb és passzívabb időszakok is, volt amikor egy-egy áttörés után rövid időre megnövekedett az elért mérföldkövek száma. A program több alakot is öltött.

4.1.1. Tanulási időszak

Az első félév alatt a program elég csekély formában volt jelen. Ebben az időszakban a fókusz a projekthez használt programozási nyelv megértésén és elsajátításán volt.

Elkészültek a program első tervei amik később támpontot adtak a fejlesztés során. Már itt előre sok elméleti problémára próbáltam megoldást keresni, ötletelni, hogy ha esetleg ezek a problémák később felmerülnének, ne érjen akkora meglepetés.

4.1.2. Szoftver alapok

A program első verziója még teljesen a *Visual Studio*-ban, egy „WinForms” felületre készült. Itt jött létre a program alapjául szolgáló algoritmus első, primitív verziója, ami a szituációk előre megadott adatokból volt képes kiszámolni az áthaladás megfelelő sorrendjét.

Ebben az állapotban még csak az alapszabályoknak megfelelően működött, nem tudta még kezelni a nem teljesen lineáris megoldásokat. A kiválasztott paraméterek alapján felállított egy sorrendet és azt adta vissza a kezelőfelületen, nem volt még vizuális megjelenítés, csak számok és szavak.

4.1.3. Unity alapok

A második félév végén nagy áttörés történt. A program átköltözött a Unity keretrendszerébe. Az eddig kifejlesztett algoritmus megkapta az interaktív felületet az első

vizuális megjelenítését. Ez már a GitHubra is felkerült.

A harmadik félévben rájöttem, hogy habár a programíráshoz már megvan a tudásom, a Unity sajátos eszközkészletét még messze nem ismerem eléggé. Tanuló anyagokat kerestem a témában, gyakorló projekteket hoztam létre, és teljesítettem egy Unity specifikus kurzust is, hogy a megfelelő szintre hozzam a tudásomat, mindemellett folyamatosan integráltam az ezalatt tanult módszereket és újdonságokat a programba.

4.1.4. Legfrissebb verzió

Az utolsó pár hónapban is sok módosításon esett átt a játék. Elkészült egy kezdetleges „feedback” rendszer ami a későbbiekben lényegesen megkönnyítette a hibakeresést és a javítást, hiszen a konzolon folyamatosan megjelentek a „real time” adatai az aktív feladatnak. Később pedig az eredetileg *placeholder*-nek szánt modellek is lecserélésre kerültek.

Az algoritmus finomodott és immár képessé vált a nem lineáris sorrendű megoldások kezelésére. Ez gyakorlatban annyit jelent, hogy az algoritmus folyamatosan újravizsgálja a szituációt minden lépés után, és megállapítja a következő helyes válaszlehetőségeket. Erről még bővebben később.

4.2. Menü

A játékban kétféle menü van. A Főmenü és a köztes játékmenü.

- Főmenü, egyszerű 3 gombbal rendelkezik:
 - Play: Elintíti a szabad játék módot.
 - Test: Elindítja a teszt játék módot.
 - Exit: Kilép a játékból, bezárja azt.
- Játékmenü
 - Next Scene: Tovább lép a következő feladatra.
 - Back to Menu: Kilép a főmenübe. Ez mindkét játékmód esetében az alap helyzetbe helyezi az előrehaladást.

4.3. Játékprogram

A projekt fő része, játék alapjaként szolgáló program, 3 fő részből áll:

- Utak

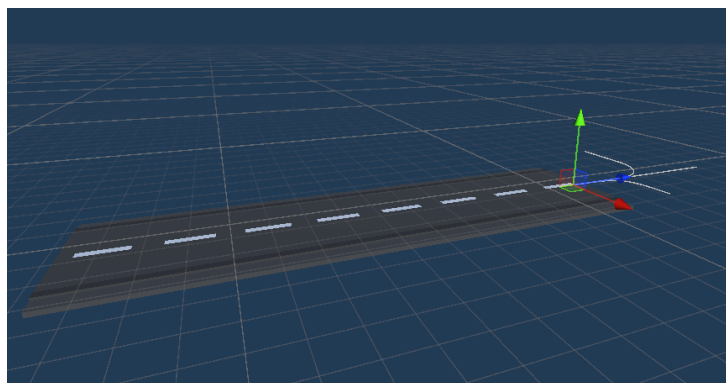
- Járművek
- Kereszteződések

Az utak és a járművek alkotják a kereszteződéseket, utóbbiak pedig az előbbiek adataiból elvégzik a szituációk létrehozásához kellő számításokat, és kezelik a megoldó algoritmust.

4.3.1. Utak

Elindítjuk a játékot, a jelenet futni kezd, a *Road* osztály az első megálló.

- Létrehozza a *Node*-okat.
 - *StartNode*: Az út objektumok jobb oldalán található Node. Ez a járművek kiindulási pontja, itt jelennek meg először.
 - *EndNode*: Az út bal oldalán található Node. Ide érkeznek a járművek, ez a kilépési pontjuk. Ez a Node határozza meg az adott jármű irányát.
- Megjelöli a felhasznált út objektumok irányát. Innen érkezik az az információ, hogy a jármű melyik utak Start- és EndNodejai közül választhat.
- Kiosztásra kerülnek az út rangok. Később ez alapján érvényesülnek az elsőbbség, és jobbkéz szabályok. Ha egy út magasabb rangú mint a szomszédja akkor az azon úton haladó jármű elsőbbséget élvez az alacsonyabb rangú úton állóval szemben. Ha az utak egyenrangúak, a jobbkéz szabály érvényesül.
- A *PlaceVehicle* metódus a *Vehicle* osztályból beérkező adatokkal egy járművet hoz létre, amit elhelyez az út *StartNode*-ján.

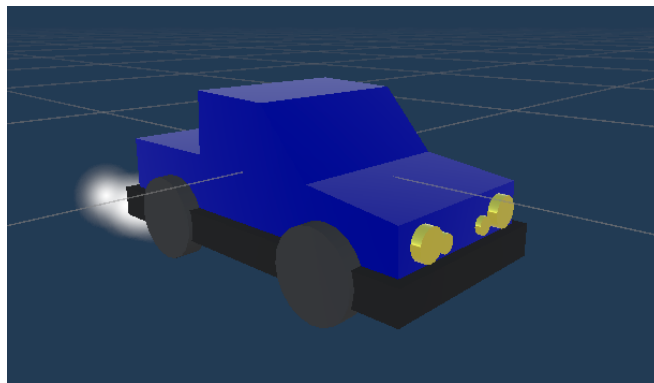


4.1. ábra. Egy út objektum felépítése

4.3.2. Járművek

A második megálló a *Vehicle* osztály.

- Létrejön egy jármű objektum, és a következő paramétereket kapja meg:
 - *Kiindulópont*: Egy olyan *StartNode* ami szabad.
 - *Irány*: Lehet előre, balra vagy jobbra.
 - *Cél*: Az alapján, hogy melyik irányba szeretne haladni, megkapja az annak megfelelő *EndNode* adatait. *EndNode* lehet közös, *StartNode* nem.
 - Jobb szomszéd: Megvizsgálja van e tőle jobbra másik jármű, hiszen egyenrangú kereszteződés esetén el kell engednie.
 - Rang: Ez a járművek rangja, a játék közben folyton változik ahogy a járművek megközelítik, majd elhagyják a kereszteződést. Ez adja meg az áthaladási sorrendet. Azért kell folyamatosan frissülnie, mert némelyik szituációban lehet több aktuálisan helyes lépés is, valamit egy lépés után lehet olyan jármű aki elveszíti a szomszédját.
 - Az animációkhoz használt komponensek is itt csatolódnak az objektumokhoz. [7]

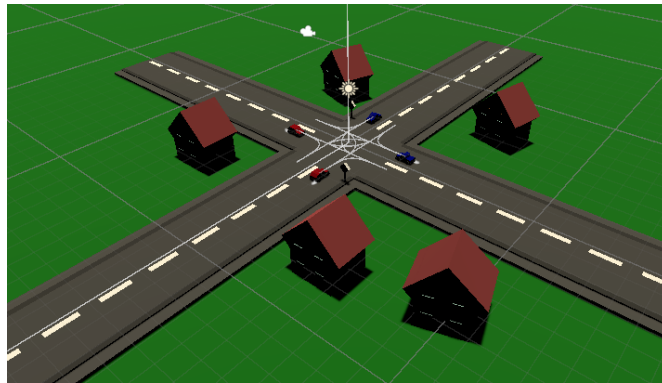


4.2. ábra. A járművek egyik modellje

4.3.3. Kereszteződések

A kereszteződéseknek két része van. Az *IntersectionBuilder* osztály és az *IntersectionSolver* osztály.

- *IntersectionBuilder*: „Megépíti” a kereszteződések. Felhasználva a kész út és jármű objektumokat és azok paramétereit, az elemeket összerakva vizualizálja a kész *Intersection* objektumot. Egy ilyen objektumnak legalább két járműből és három útszakaszból kell állnia, hogy megfelelő kereszteződést alkosson.
- *IntersectionSolver*:
 - Kezeli a megoldó algoritmust, a folyamatosan frissülő áthaladási sorrendet és az alapján Log-ol a konzolra is.
 - Kezeli a rövid és hosszútávú eredményeket a pontszámrendszerrel a szabad játék és a teszt játék esetében is.



4.3. ábra. Egy éppen aktív kereszteződés

5. fejezet

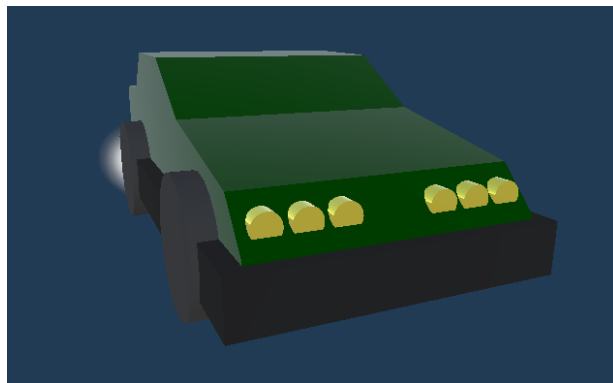
Fejlesztési lehetőségek

Rengeteg potenciál van még a programban. Sok olyan ötletem volt és van ami még nem valósult meg, de elméletben már létezik.

5.1. Textúrák és modellek

A projekt alatt sokkal több figyelmet fordítottam a program működési folyamatainak megbízhatóságára mint a megjelenésére, így van még pár olyan része a játéknak amire ráférne egy ráncfelvarrás:

- Autók: A játék kis szereplői, a közlekedésben résztvevő járművek habár aranyosak a maguk alacsony bonyolultságú kinézetükkel, sokat hozzátenne a játékélményhez az alaposabban kidolgozott autó modellek alkalmazása. A fejlesztendő textúrák és modellek közül ez számomra a legfontosabb, és ez egy hozzáértő új csapattag bevonásával könnyen megvalósítható lenne.



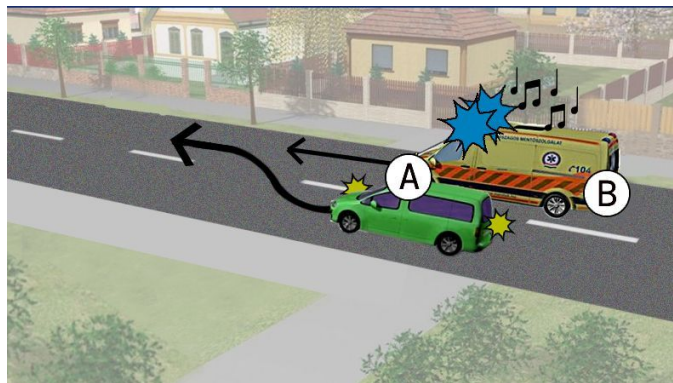
5.1. ábra. A járművek egyik modellje, amit lehetne fejleszteni

- Gombok és feliratok: A játék legtöbb felirata és gombjai egyszerű kinézettel rendelkeznek. Ezeknek a cseréje szintén nagyon feldobná a játékot és jobb esztétikai élményt adna.
- Környezet: A játékkörnyezet legalább annyira fontos mint a résztvevő autóké. A környezet kidolgozottsága egyeznie kell a járművekével, hogy az autók ne tűnjenek ki a helyzetből jobban mint kellene.

5.2. Új pályák és szituációk

Szeretném, hogy a játék ne legyen túlságosan egyoldalú. Ennek az eléréséhez a jövőben új pályákat tervezek hozzáadni amik egy fokkal feljebb emelik az alap kereszteződés alapú pályákat.

A kereszteződések mellett a következő lépés a kétirányú úttestek, körforgalmak és lámpás kereszteződések, esetleg további résztvevő járművek integrálása lesz.



5.2. ábra. KRESZ feladat mentőautóval [13]

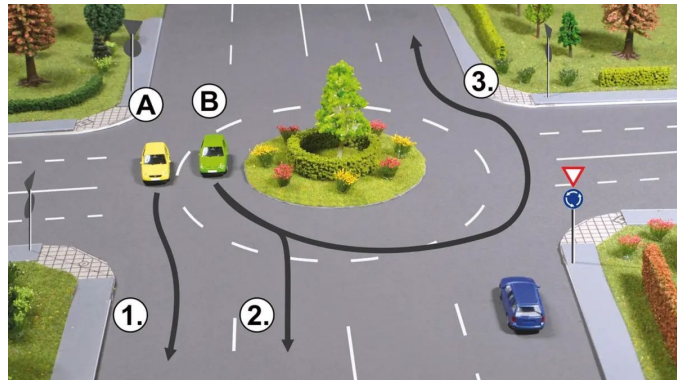
A nehézségi szintet tovább emelve, a járműveknek *Collidert* adva az áthaladási animációkkal egyszerű baleseteket is szimulálhatnánk, ami türelemre és odafigyelésre is tanítaná a játékost.

5.3. Új szabályrendszerek

Az új kétirányú, körforgalom és lámpás kereszteződések beépítésével új szabályrendszerekre is szükség lesz.

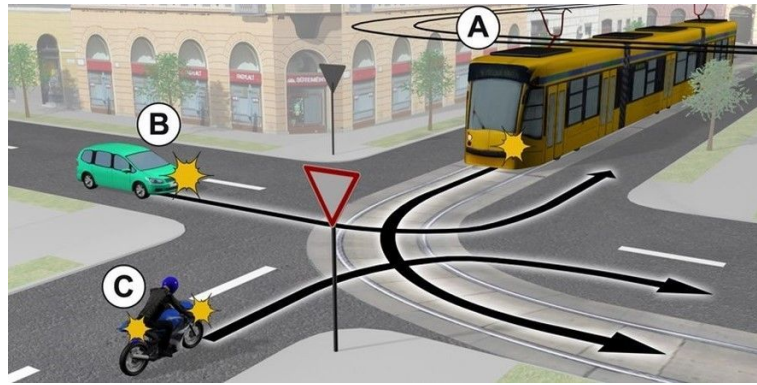
- Kétirányú úttest: Az ilyen típusú pályához kell egy olyan algoritmus ami tudja szimulálni az előzés, kikerülés, úttest elhagyása és a megkülönböztető jelzéssel haladó jármű érkezését, és az ezek által létrehozott szimulációban a kereszteződésekhez hasonlóan áthaladási sorrendet és megoldást generálni.

- Körforgalom: A Körforgalom egy egyszerű de annál furfangosabb és fontos része a közúti közlekedésnek. A beépítéséhez egy külön elsőbbség adási rendszer, és az irányjelzés és az azt figyelő algoritmus is része kell, hogy legyen.

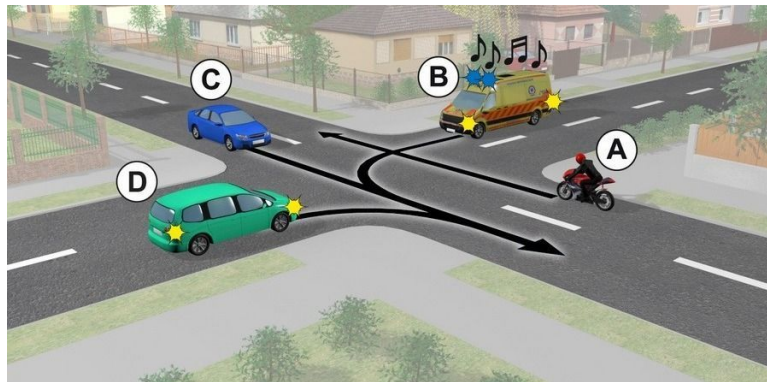


5.3. ábra. KRESZ feladat körforgalommal [14]

- Lámpás kereszteződés: A lámpás kereszteződés a mostanihoz hasonló de egy lépcsőfokkal ellenőrzöttebb rendszer, viszont fontosnak érzem, hogy a későbbiekben ez is a része legyen a programnak.
- Új járművek: A KRESZ-ben gyakran előfordulnak speciális járművek és résztvevők amikre a forgalom különös figyelemmel vagy akár konkrét szabályokkal reagál. Ilyen például:
 - Vonat: Vasúti átkelőhely.
 - Villamos: Kereszteződés villamossínnel, opcionális érkező villamossal.
 - Megkülönböztető jelzés: Rendőr, mentő, tűzoltó! A kereszteződésekbe és többi pályafarmába is opció.
 - Gyalogos: Sok esetben nem teljesen egyértelmű, hogy egy gyalogosnak mikor van elsőbbsége az autókkal szemben, és ez könnyen bezavarhat egy-egy szituációban, így extra nehezítésként a gyalogosok jelenlétét is lehetne adni a feladathoz.



5.4. ábra. KRESZ feladat villamossal [11]



5.5. ábra. KRESZ feladat mentőautóval [12]

5.4. Szabálylista

A játékon belül egy olyan felület, ahol a játék során fellelhető közlekedési szabályokat magyarázattal és illusztrációval vannak feltüntetve. Így ezeket a szabályokat a programon belül tanulmányozhatja a felhasználó, ha valamelyiknek szeretne utánanézni esetleg egy elrontott feladat után.

5.5. Testreszabható feladatok

Még az első bővítési ötleteim között volt a szituációk testreszabhatósága. Ez azt jelentené, hogy egy külön játékmódban a felhasználó beállíthatja, hogy:

- Milyen kereszteződés vagy közlekedési helyzet legyen a szituáció alapja.
- Milyen és hány résztvevője legyen az adott feladatnak.
- Az adott résztvevők milyen irányban haladjanak tovább, rendelkezzenek e bármilyen plusz joggal.

A beállítások véglegesítése után a játékos megoldhatja a saját szituációját aztán pedig új feladat létrehozásába kezdhet.

6. fejezet

Tesztelés

A programozás egyik elengedhetetlen része a tesztelés. A projektem fejlesztése során kétféle tesztelési módszert alkalmaztam: Manuális és egységtesztek, másik nevén *Unit tesztek*.

6.1. Manuális

A manuális tesztelés során olyan funkciókat vizsgálok, mint például a bemenet, megjelenítés, vagy a menü különböző gombjainak működése. A fejlesztés közben folyamatosan csináltam manuális teszteket. Minden új funkciót rögtön tesztelni kell, hogy megfelelően működik, így nagyon gyorsan kiszűrhetőek az esetleges hibák.

Később már végeztem nagyobb, teljes rendszer szintű manuális teszteket is, ahol egyben az egész játékot teszteltem, és nem csak a legfrissebb funkciókat. Ez azért fontos mert egy hosszabb fejlesztés után ha egy-egy funkció jól is működik, nem akadályozhatják egymást.



6.1. ábra. Manuális tesztelés (Teszt játék, nem elég pont)

A manuális tesztek eredményei:

- A szabad játék és a teszt játék pontrendszere is megfelelően működik és minden esetben helyesen írja ki az aktuális értéket.
- Az animációk megfelelően és simán futnak végig.
- A megoldó és sorrend felállító algoritmusok megfelelően működnek. Felismerik az esetleges egyenrangúságot a járművek esetében és tudják kezelni.
- A Főmenü és a játékmenü gombjai is megfelelően működnek.

6.2. Egységteszt

Az egységtesztek olyan tesztek amiket a programunk adatokkal és változókkal foglalkozó részére írunk. Ezek a tesztek automatizáltak és azt tudjuk velük ellenőrizni, hogy a szoftver teljesíti-e a feladatait.

A Unity keretrendszerben ezt egy külön package telepítésével tehetjük meg, a *Test Framework*-el. A kezeléséhez megnyitjuk a tesztelő felületet, azaz a *Test Runner*-t. Két opciónk van:

- Play Mode: Itt olyan teszteket tudunk csinálni amelyek futási időben dolgoznak. Ilyenkor a keretrendszer elindítja a játékot és úgy futtatja a teszteket.
- Edit Mode: Ebben a módban azok a kódrészek tesztelhetők amelyek nem foglalkoznak a futásidő alatt aktív objektumokkal.

Összegzés

A szakdolgozat végére úgy érzem, hogy az elképzelésem megvalósulni látszanak a kész programban. Eleinte úgy gondoltam, hogy sokkal egyszerűbb lesz, viszont ahogy haladtam előre a fejlesztésben, az újabb és újabb akadályok megmutatták mennyi belefektetett energiát is igényel egy ilyen program elkészítése. Rengeteg ötletem lenne még a program fejlesztésére, hogy milyen plusz funkciókat és elemeket tudnék beleépíteni a játékba. Ennek ellenére elégedett vagyok a program mostani állapotával is, hiszen mindig jó ha van hova fejlődni.

Tanulás szempontjából nagyon hasznosnak éreztem ezt a projektet. Rengeteg önfejlesztésen kellett átesnem, hogy elsajátítsam azt a tudást ami egy ilyen program fejlesztéséhez elengedhetetlen. Az 5. fejezetben leírtakból szeretnék a lehető legtöbbet valamilyen módon a közeljövőben beépíteni a programba, ezáltal tovább fejlesztve és gyakorlással elmélyítve a tudásomat.

Véleményem szerint ebből a programból nagyon hasznos eszköz válhat azok számára akik vezetni szeretnének tanulni és kell egy kis segítség a szabályok megértésében. A közlekedési szabályok megértése és gyakorlatban való alkalmazása rengeteg figyelmet és gyakorlást igényel, és nagyon fontos, hogy mindenki aki résztvesz a közlekedésben megfelelően fel legyen készítve a szabályok használatára.

Irodalomjegyzék

- [1] WIKIPÉDIA: *Id Software*, https://en.wikipedia.org/wiki/Id_Software, Megtekintve: 2026.04.05.
- [2] WIKIPÉDIA: *Unreal Engine*, https://en.wikipedia.org/wiki/Unreal_Engine, Megtekintve: 2026.04.05.
- [3] WIKIPÉDIA: *Godot (game engine)*, [https://en.wikipedia.org/wiki/Godot_\(game_engine\)](https://en.wikipedia.org/wiki/Godot_(game_engine)), Megtekintve: 2026.04.05.
- [4] WIKIPÉDIA: *Unity (game engine)*, [https://en.wikipedia.org/wiki/Unity_\(game_engine\)](https://en.wikipedia.org/wiki/Unity_(game_engine)), Megtekintve: 2026.04.05.
- [5] WIKIPÉDIA: *Unity Technologies*: https://en.wikipedia.org/wiki/Unity_Technologies, Megtekintve: 2026.04.06.
- [6] UNITY DOCUMENTATION: *Introduction to components*: <https://docs.unity3d.com/Manual/Components.html>, Megtekintve: 2026.04.08.
- [7] UNITY DOCUMENTATION: *MonoBehaviour*: <https://docs.unity3d.com/ScriptReference/MonoBehaviour.html>, Megtekintve: 2026.04.08.
- [8] WIKIPÉDIA: *KRESZ*: <https://hu.wikipedia.org/wiki/KRESZ>, Megtekintve: 2026.04.18.

Ábrajegyzék

- [9] KRESZ VIZSGA: *Három pontos feladat*, <https://szuperjogsi.hu/images/slide/3/Bogi110.jpg>, Letöltve: 2026.04.20.
- [10] KRESZ TÁBLÁK: *Közlekedési táblák*, https://vezess2.p3k.hu/app/uploads/2007/02/2123_19519_1000x700.jpg, Letöltve: 2026.04.20.
- [11] KRESZ VIZSGA: *Három pontos feladat villamossal*, <https://www.beol.hu/helyi-eletstilus/2025/07/kresz-teszt-villamos-elsobbseg>, Letöltve: 2026.04.20.
- [12] KRESZ VIZSGA: *Három pontos feladat mentőautóval*, <https://www.feol.hu/helyi-kozelet/2024/12/kresz-teszt-mentoauto>, Letöltve: 2026.04.20.
- [13] KRESZ VIZSGA: *KRESZ feladat mentőautóval*, https://www.szon.hu/helyi-kozelet/2025/07/megelozheto-a-mento-a-legtobben-bebukjak-ezt-a-kerdest#google_vignette, Letöltve: 2026.04.20.
- [14] KRESZ VIZSGA: *Három pontos feladat körforgalommal*, <https://www.beol.hu/helyi-eletstilus/2026/03/kresz-teszt-korforgalom>, Letöltve: 2026.04.20.
- [15] DOOM: *DOOM képernyőkép*, https://upload.wikimedia.org/wikipedia/en/thumb/d/de/Doom_ingame_1.png/250px-Doom_ingame_1.png, Letöltve: 2026.04.20.
- [16] UNREAL ENGINE: *Unreal Engine kezelőfelület*, <https://cdn2.unrealengine.com/ue5-modeling-mode-tech-blog-sandsculpting-2844x1559-1885809c9ebd.JPG>, Letöltve: 2026.04.20.
- [17] GODOT ENGINE: *Godot Engine kezelőfelület*, <https://upload.wikimedia.org/wikipedia/commons/e/e3/Godot3.4.png>, Letöltve: 2026.04.20.
- [18] UNITY ENGINE: *Unity kezelőfelület*, <https://unity.com/releases/2019-3/editor-tools>, Letöltve: 2026.04.20.

Nyilatkozat

Alulírott, büntetőjogi felelősségem tudatában kijelentem, hogy az általam benyújtott, című szakdolgozat önálló szellemi termékem. Amennyiben mások munkáját felhasználtam, azokra megfelelően hivatkozom, beleértve a nyomtatott és az internetes forrásokat is.

Aláírással igazolom, hogy az elektronikusan feltöltött és a papíralapú szakdolgozatom formai és tartalmi szempontból mindenben megegyezik.

Eger, 2026. március 14.

aláírás

A szakdolgozat megírása után ezt a nyilatkozatot kell a végéhez csatolni a következő módon:

1. A `nyilatkozat` mappában a `nyilatkozat.tex` fájlt töltsse ki és fordítsa le pdf-be!
2. A `nyilatkozat.pdf` fájlt nyomtassa ki, majd írja alá!
3. Szkennelje be pdf formátumba, majd ezzel írja felül a `nyilatkozat` mappában a `nyilatkozat.pdf` fájlt!
4. A szakdolgozat forrásfájljában legyen betöltve a `pdfpages` csomag. Az utolsó két sor legyen az alábbi:

```
\includepdf{nyilatkozat/nyilatkozat.pdf}  
\end{document}
```

5. Ezután fordítsa le a szakdolgozatot pdf-be!